HACKMERIDIAN 2026 · STELLAR COMPOSED

Blendlord

Your rental deposit can make you money.

Yield-bearing rental deposits on Stellar · Soroban escrow + Blend + DeFindex + a risk model · live on testnet

Today, a deposit is money doing nothing

It earns nothing

One or two months of rent, locked for years. Not a cent of interest for the tenant.

One side holds it

The landlord keeps the cash, so the move-out is a negotiation where one side already has the money.

Escrow is not enough

Escrows fix trust, but the money still sits idle. Nobody makes it work safely.

A neutral contract holds it and puts it to work

1

Offer

The landlord deploys the agreement with its terms, signed on-chain.

2

Fund

The tenant accepts, chooses how much to invest and where, and locks USDC.

3

Earn

Up to 80% earns yield in Blend and DeFindex. At least 20% stays liquid.

4

Settle

Agreed repairs to the landlord first; deposit and yield to the tenant.

Disagree? Nothing moves. A neutral arbiter decides, and can never award more than the landlord asked.

Each side decides what is theirs

Landlord

- Amount and lease dates
- Neutral arbiter
- A small share of the yield: 3% by default

Tenant

- How much to invest, up to 80%
- Where: Blend direct or the DeFindex reserve
- Keeps at least 95% of the yield

The contract

- At least 20% always liquid
- Landlord share capped at 5%, of the yield only
- Both signatures, or the arbiter

Composed on Stellar

Soroban escrow

Terms, state machine, waterfall, timeouts, 80% cap

Blend

Direct supply: approve + submit_with_allowance

DeFindex

Our own vault: a shared reserve across agreements

Risk model

Recommends the share and rebalances the reserve

One tenant signature funds both venues: the contract authorizes the vault's pull itself with authorize_as_current_contract. USDC throughout, Freighter in the app.

How much can safely earn yield?

A payout is delayed only if the deduction exceeds the liquid buffer **and** the venue can't pay that day:

$$P(\text{delay}) = P(D > 1 - w) \cdot q \leq 0.5\%$$

So the buffer is a quantile of the deduction distribution, like Value at Risk. Checked with a 2-million-lease Monte Carlo.

Without early unwind (q = 25%)

51%

safe share to invest

With unwind() in the last 30 days (q = 2%)

80%

safe share: about 60% more yield, same risk

DeFindex: a reserve the risk model manages

Pool crowded

89.3%

Blend utilization read live → target 47% invested → **invest 18.80 USDC**

Move-out coming

10 days

Glide path → **unwind 18.80 USDC**, reserve back to 100% liquid

Platform fee

1%

of the vault's yield, on-chain (paused on testnet)

Both moves executed on testnet by **rebalance.py** through the vault's rebalance-manager role. On testnet the strategy is the same Blend pool: the value is the managed, shared reserve, not a second yield source.

Built, tested and live on testnet

48

unit tests, including real-auth and illiquid-venue cases

2

independent reviews; every finding fixed with a regression test

4

full flows on testnet: happy path, unwind, dispute + arbiter, Blend + DeFindex

0.003

XLM per action, about a tenth of a cent

Nobody can hold the deposit hostage: timeouts for a silent landlord, tenant or arbiter, and a liquid buffer that pays the landlord first even if a venue is illiquid.

Everyone gets something, only from yield

Tenant

≥ 95%

of the yield, and the whole deposit minus agreed repairs

Landlord or agency

3%

of the yield by default, capped at 5%: the reason to offer it

Blendlord

1%

fee on the reserve's yield (paused on testnet)

Go-to-market: letting agencies and student residences, which handle many deposits and pick the custody option for their tenants.

Next

- Factory contract with an allow-list of pools, vaults and assets
- Multi-strategy reserve, allocated with the same risk budget
- Calibrate the model with real deduction data from agencies
- Arbitration via multisig or a partner; custody on existing escrow rails

Others hold deposits safely. We make them work.

blendlord.netlify.app · github.com/ddortabencomo/yield-bearing-rental-deposit